LEZIONE 8 - Operatori ed Espressioni

Durata: 4 ore | Teoria: 2h | Esercizi: 2h

PARTE TEORICA (2 ore)

8.1 Introduzione agli Operatori (10 min)

Cos'è un Operatore?

Un **operatore** è un simbolo che indica un'operazione da eseguire su uno o più **operandi**.

Esempio:

```
int risultato = 5 + 3;
//              ↑ ↑ ↑
//              | | └── operando 2
//              | └──── operatore
//              └────── operando 1
```

Classificazione per Numero di Operandi

1. Operatori Unari (1 operando)

```
-x      // negazione
++x     // incremento
!x      // NOT logico
```

2. Operatori Binari (2 operandi)

```
a + b   // addizione
a > b   // maggiore di
a && b  // AND logico
```

3. Operatori Ternari (3 operandi)

```
condizione ? valore_se_vero : valore_se_falso
```

Classificazione per Tipo di Operazione

- **Aritmetici**: +, -, *, /, %
- **Relazionali**: <, >, <=, >=, ==, !=
- **Logici**: &&, ||, !
- **Bitwise**: &, |, ^, ~, <<, >>
- **Assegnazione**: =, +=, -=, *=, /=, %=
- **Incremento/Decremento**: ++, –
- **Altro**: sizeof, ternario (?:), virgola (,)

8.2 Operatori Aritmetici (esempi e note)

Operatori Base

1. Addizione (+)

```
int a = 5 + 3;          // 8
float b = 2.5 + 1.3;    // 3.8
int c = -10 + 15;       // 5
```

2. Sottrazione (-)

```
int a = 10 - 3;          // 7
float b = 5.5 - 2.3;     // 3.2
int c = 0 - 5;           // -5
```

3. Moltiplicazione (*)

```
int a = 4 * 3;           // 12
float b = 2.5 * 4.0;     // 10.0
int c = -3 * 5;          // -15
```

4. Divisione (/)

```
int a = 10 / 3;          // 3 (divisione intera!)
float b = 10.0 / 3.0;    // 3.333...
float c = 10.0 / 3;      // 3.333... (uno float basta)
int d = 10 / 0;          // ERRORE! Undefined behavior
```

IMPORTANTE: Divisione intera vs reale

```
int x = 7, y = 2;

int r1 = x / y;           // 3 (troncamento)
float r2 = x / y;         // 3.0 (divisione intera, poi convertito)
float r3 = (float)x / y;  // 3.5 (divisione reale) ✓
```

5. Modulo (%) - Resto della Divisione

```
int a = 10 % 3;          // 1 (10 = 3*3 + 1)
int b = 15 % 4;          // 3 (15 = 4*3 + 3)
int c = 20 % 5;          // 0 (divisione esatta)
int d = 7 % 2;           // 1 (pari=0, dispari=1)

// float e = 10.5 % 3.2; // ERRORE! % solo per interi
```

Usi pratici del modulo:

```
// Verificare se pari/dispari
if (n % 2 == 0) {
    printf("Pari\n");
}

// Ciclare array (wraparound)
int indice = (i + 1) % array_size;

// Estrarre ultima cifra
int ultima_cifra = numero % 10;

// Estrarre cifre
int centinaia = numero / 100;
int decine = (numero / 10) % 10;
int unita = numero % 10;
```

Operatori Unari

Negazione (-)

```
int x = 5;
int y = -x;      // -5
int z = -(-x);   // 5
```

Positivo (+) (raramente usato)

```
int x = +5;      // equivalente a: int x = 5;
```

Esempi Completi

Esempio 1: Calcolo media

```c
int voto1 = 8, voto2 = 7, voto3 = 9;
float media = (voto1 + voto2 + voto3) / 3.0;  // 8.0
// Nota: 3.0 per divisione reale!
```

Esempio 2: Conversione ore

```c
int secondi_totali = 3665;
int ore = secondi_totali / 3600;      // 1
int minuti = (secondi_totali % 3600) / 60;  // 1
int secondi = secondi_totali % 60;    // 5
// Risultato: 1h 1m 5s
```

Esempio 3: Scambio senza variabile temporanea

```c
int a = 5, b = 3;
a = a + b;  // a = 8
b = a - b;  // b = 5
a = a - b;  // a = 3
// Ora: a = 3, b = 5
```

8.3 Incremento/Decremento

Incremento (++)

Post-incremento (x++) - Usa il valore PRIMA, poi incrementa

```c
int x = 5;
int y = x++;  // y = 5, x = 6
```

Pre-incremento (++x) - Incrementa PRIMA, poi usa il valore

```c
int x = 5;
int y = ++x;  // y = 6, x = 6
```

Confronto:

```c
int a = 5, b = 5;

printf("%d\n", a++);  // Stampa: 5, poi a diventa 6
printf("%d\n", a);    // Stampa: 6

printf("%d\n", ++b);  // b diventa 6, poi stampa: 6
printf("%d\n", b);    // Stampa: 6
```

Decremento (–)

Post-decremento (x–)

```c
int x = 5;
int y = x--;  // y = 5, x = 4
```

Pre-decremento (–x)

```c
int x = 5;
int y = --x;  // y = 4, x = 4
```

Quando Usare Pre vs Post

In espressioni semplici, sono equivalenti:

```c
x++;    // Incrementa x
++x;    // Incrementa x (stesso effetto)
```

In espressioni complesse, differiscono:

```c
int i = 0;
int array[10];

array[i++] = 5;  // array[0] = 5, poi i diventa 1
array[++i] = 7;  // i diventa 2, poi array[2] = 7
```

Nei cicli (anticipazione):

```c
// Equivalenti:
for (i = 0; i < 10; i++)   // ...
for (i = 0; i < 10; ++i)   // ...

// Preferenza: ++i (leggermente più efficiente in C++)
```

Errori Comuni

ERRORE: Usare ++ su espressioni

```c
int x = 5, y = 3;
(x + y)++;     // ERRORE! Non è un l-value
++(x + y);     // ERRORE!
```

ERRORE: Doppio incremento

```c
int x = 5;
int y = x++ + ++x;  // UNDEFINED BEHAVIOR! Evitare!
```

Best Practice: - Usa ++ e – su righe separate quando possibile - Evita espressioni complesse con effetti collaterali - In caso di dubbio, scrivi più chiaramente

---

8.4 Relazionali

Gli operatori **relazionali** confrontano due valori e restituiscono **1** (vero) o **0** (falso).

Operatori di Confronto

1. Uguale a (==)

```c
int a = 5, b = 5, c = 3;
int r1 = (a == b);  // 1 (vero)
int r2 = (a == c);  // 0 (falso)
```

ATTENZIONE: Non confondere == (confronto) con = (assegnazione)!

```c
int x = 5;
if (x = 10) {        // BUG! Assegna 10 a x, sempre vero
    printf("Oops\n");
}

if (x == 10) {       // Corretto: confronta
    printf("OK\n");
}
```

2. Diverso da (!=)

```c
int a = 5, b = 3;
int r = (a != b);    // 1 (vero, sono diversi)
```

3. Maggiore di (>)

```c
int a = 10, b = 5;
int r = (a > b);     // 1 (vero)
```

4. Minore di (<)

```c
int a = 3, b = 7;
int r = (a < b);     // 1 (vero)
```

5. Maggiore o uguale (>=)

```c
int a = 5, b = 5;
int r1 = (a >= b);   // 1 (vero, sono uguali)
int r2 = (a >= 3);   // 1 (vero, 5 >= 3)
```

6. Minore o uguale (<=)

```c
int a = 5, b = 10;
int r = (a <= b);    // 1 (vero)
```

Comparazione con Float

PROBLEMA: Float hanno errori di arrotondamento!

```c
float a = 0.1 + 0.2;
float b = 0.3;

if (a == b) {                    // Probabilmente FALSO!
    printf("Uguali\n");
}

// SOLUZIONE: Usa epsilon (tolleranza)
#include <math.h>
#define EPSILON 0.00001

if (fabs(a - b) < EPSILON) {     // Corretto
    printf("Uguali (entro tolleranza)\n");
}
```

Confronto Caratteri

```c
char c1 = 'A', c2 = 'B';

if (c1 < c2) {  // Vero (confronta codici ASCII: 65 < 66)
    printf("A viene prima di B\n");
}

// Controlli utili
char ch = 'g';
if (ch >= 'a' && ch <= 'z') {
    printf("Lettera minuscola\n");
}
```

Espressioni Relazionali

```c
int eta = 25;
char tessera = 'S';  // S = ha tessera, N = no
int prezzo_pieno = 10;

// Sconto solo se: (età < 18 o età > 65) E ha tessera
if ((eta < 18 || eta > 65) && tessera == 'S') {
    int prezzo = prezzo_pieno / 2;
    printf("Prezzo ridotto: %d€\n", prezzo);
} else {
    printf("Prezzo pieno: %d€\n", prezzo_pieno);
}
```

8.5 Logici

Gli operatori **logici** combinano espressioni booleane.

AND Logico (&&)

Vero solo se ENTRAMBI gli operandi sono veri.

Tabella verità:

```
A     B     A && B
------------------------
0     0       0
0     1       0
1     0       0
1     1       1
```

Esempi:

```c
int a = 5, b = 10;

if (a > 0 && b > 0) {            // Vero
    printf("Entrambi positivi\n");
}

if (a > 0 && b < 5) {            // Falso (b < 5 è falso)
    printf("Non stampato\n");
}

// Controllo range
int x = 50;
if (x >= 0 && x <= 100) {
    printf("x è tra 0 e 100\n");
}
```

Short-circuit evaluation:

```c
int x = 0;
if (x != 0 && (10 / x) > 2) {    // Sicuro! Se x=0, non valuta 10/x
    printf("OK\n");
}
```

OR Logico (||)

Vero se ALMENO UNO degli operandi è vero.

Tabella verità:

```
A     B     A || B
------------------------
0     0       0
0     1       1
1     0       1
1     1       1
```

Esempi:

```c
char ch = 'A';

if (ch == 'a' || ch == 'A') {    // Vero
    printf("È la lettera A\n");
}

// Controllo multiplo
int giorno = 6;
if (giorno == 6 || giorno == 7) {
    printf("È weekend!\n");
}

// Validazione
int voto = 110;
if (voto < 0 || voto > 100) {
    printf("Voto non valido\n");
}
```

Short-circuit:

```c
if (x == 0 || (10 / x) > 2) {    // Se x=0, non valuta 10/x
    printf("OK\n");
}
```

NOT Logico (!)

Inverte il valore booleano.

Tabella verità:

```
A     !A
-----------
0      1
1      0
```

Esempi:

```c
int x = 0;
int y = !x;      // y = 1 (NOT di 0)

int a = 5;
int b = !a;      // b = 0 (NOT di qualsiasi numero != 0)

// Verifica negativa
if (!(x > 10)) {
    printf("x NON è maggiore di 10\n");
}

// Equivalente a:
if (x <= 10) {
    printf("x è minore o uguale a 10\n");
}
```

Combinazioni Complesse

```c
int eta = 25;
char tessera = 'S';  // S = ha tessera, N = no
int prezzo_pieno = 10;

// Sconto solo se: (età < 18 o età > 65) E ha tessera
if ((eta < 18 || eta > 65) && tessera == 'S') {
    int prezzo = prezzo_pieno / 2;
    printf("Prezzo ridotto: %d€\n", prezzo);
} else {
    printf("Prezzo pieno: %d€\n", prezzo_pieno);
}
```

Leggi di De Morgan

Equivalenze utili:

```
!(A && B)  ≡  !A || !B
!(A || B)  ≡  !A && !B

// Esempio:
// "Non è vero che piove E fa freddo"
// equivale a:
// "Non piove O non fa freddo"
```

Valori di Verità in C

In C, **qualsiasi valore diverso da 0 è considerato vero**.

```c
int a = 5;
if (a) {              // Vero! (5 != 0)
    printf("a è vero\n");
}

int b = 0;
if (b) {              // Falso
    printf("Non stampato\n");
}

if (!b) {             // Vero! (NOT di 0)
    printf("b è falso\n");
}
```

8.6 Assegnazione composta

Gli operatori **composti** combinano operazione e assegnazione.

Operatori Disponibili

```
+=    -=    *=    /=    %=    &=    |=    ^=    <<=    >>=
```

Esempi Base

Addizione composta (+=)

```c
int x = 10;
x += 5;       // Equivalente a: x = x + 5;
// x = 15
```

Sottrazione composta (-=)

```c
int x = 20;
x -= 3;       // Equivalente a: x = x - 3;
// x = 17
```

Moltiplicazione composta (*=)

```c
int x = 4;
x *= 3;       // Equivalente a: x = x * 3;
// x = 12
```

Divisione composta (/=)

```c
int x = 20;
x /= 4;       // Equivalente a: x = x / 4;
// x = 5
```

Modulo composto (%=)

```c
int x = 17;
x %= 5;       // Equivalente a: x = x % 5;
// x = 2
```

Confronto Forme

```c
// Forma lunga
x = x + 10;
y = y * 2;
z = z / 3;

// Forma compatta (preferita)
x += 10;
y *= 2;
z /= 3;
```

Vantaggi

1. Più conciso

```
lunghissimo_nome_variabile = lunghissimo_nome_variabile + 5;
lunghissimo_nome_variabile += 5;  // Più leggibile
```

2. Più efficiente (teoricamente)

```
array[funzione_complessa(x)] = array[funzione_complessa(x)] + 1;
array[funzione_complessa(x)] += 1;  // Valuta indice una sola volta
```

3. Meno errori

```
x = y + 1;    // Intendevi x = x + 1? Errore comune
x += 1;       // Chiaro!
```

Esempi Pratici

Contatore:

```
int contatore = 0;
contatore += 1;  // oppure: contatore++
```

Accumulo:

```
float totale = 0.0;
for (int i = 0; i < n; i++) {
    totale += array[i];
}
```

Moltiplicazione per potenze di 2:

```
x = x << 3;  // x = x * 8  (più veloce di x *= 8)
x = x >> 2;  // x = x / 4  (più veloce di x /= 4)
```

8.7 Ternario

L'**operatore ternario** (?:) è l'unico operatore con 3 operandi.

Sintassi

```
condizione ? valore_se_vero : valore_se_falso
```

Esempi Base

Assegnazione condizionale:

```
int a = 10, b = 20;
int max = (a > b) ? a : b;   // max = 20

// Equivalente a:
int max;
if (a > b) {
    max = a;
} else {
    max = b;
}
```

In printf:

```
int eta = 17;
printf("Sei %s\n", (eta >= 18) ? "maggiorenne" : "minorenne");
```

Calcolo assoluto:

```c
int x = -5;
int abs_x = (x < 0) ? -x : x;  // abs_x = 5
```

Ternario Annidato

Possibile ma sconsigliato:

```c
int voto = 85;
char *valutazione = (voto >= 90) ? "Ottimo" :
                    (voto >= 80) ? "Buono" :
                    (voto >= 70) ? "Discreto" :
                    (voto >= 60) ? "Sufficiente" : "Insufficiente";
```

Meglio usare if-else per chiarezza:

```c
if (voto >= 90) {
    valutazione = "Ottimo";
} else if (voto >= 80) {
    valutazione = "Buono";
} // ...
```

Quando Usare il Ternario

✓ BUONO (semplice, leggibile):

```c
int max = (a > b) ? a : b;
int segno = (x < 0) ? -1 : 1;
printf("%d\n", (n % 2 == 0) ? 0 : 1);
```

✗ CATTIVO (complesso, illeggibile):

```c
int x = (a > b) ? (c > d) ? (e > f) ? 1 : 2 : 3 : 4;  // Confuso!
```

8.8 Precedenza e Associatività

La **precedenza** determina l'ordine di valutazione degli operatori.

Tabella Precedenza (dal più alto al più basso)

Livello	Operatori	Descrizione	Associatività
1	() [] . ->	Parentesi, accesso	L→R
2	! ~ ++ – + - * & (cast) sizeof	Unari	R→L
3	* / %	Moltiplicazione, divisione, modulo	L→R
4	+ -	Addizione, sottrazione	L→R
5	<< >>	Shift bit	L→R
6	< <= > >=	Relazionali	L→R
7	== !=	Uguaglianza	L→R
8	&	AND bitwise	L→R
9	^	XOR bitwise	L→R
10	|	OR bitwise	L→R
11	&&	AND logico	L→R
12	||	OR logico	L→R
13	?:	Ternario	R→L
14	= += -= *= /= %= etc.	Assegnazione	R→L
15	,	Virgola	L→R

Esempi Precedenza

Aritmetica:

```c
int x = 2 + 3 * 4;        // x = 14 (non 20!)
// Equivalente a: 2 + (3 * 4)

int y = 10 - 4 / 2;       // y = 8 (non 3!)
// Equivalente a: 10 - (4 / 2)
```

Con parentesi (override precedenza):

```c
int x = (2 + 3) * 4;      // x = 20
int y = (10 - 4) / 2;     // y = 3
```

Relazionali e logici:

```c
int a = 5, b = 10, c = 3;

if (a < b && b > c) {     // OK: relazionali prima di &&
    printf("Vero\n");
}

// Senza &&, avremmo bisogno di parentesi:
int result = a < b > c;  // Confuso! Usa parentesi
int result = (a < b) && (b > c);  // Chiaro
```

Assegnazione:

```c
int x, y, z;
x = y = z = 5;            // Associatività R→L
// Equivalente a: x = (y = (z = 5));
```

Associatività

Left-to-Right (L→R):

```c
int x = 10 - 5 - 2;       // (10 - 5) - 2 = 3
int y = 20 / 4 / 2;       // (20 / 4) / 2 = 2
```

Right-to-Left (R→L):

```c
int x = 5;
x = x + 1;                // R→L per =
// Non confondere con: (x = x) + 1

int *p = &x;              // R→L per unari
```

Best Practices

1. Usa parentesi quando in dubbio:

```c
// Poco chiaro
if (a && b || c && d) { }

// Chiaro
if ((a && b) || (c && d)) { }
```

2. Non affidarti troppo alla precedenza:

```c
// Funziona ma poco leggibile
int result = a * b + c / d - e;

// Meglio
int temp = b * c;
int result = a + (temp / d) - (e % f);
```

3. Una operazione per riga (quando complesso):

```
// Difficile
int result = a * b + c / d - e;

// Più chiaro
int prod = a * b;
int quo = c / d;
int result = prod + quo - e;
```

8.9 Bitwise (panoramica)

Gli operatori **bitwise** lavorano sui singoli bit.

Operatori Disponibili

```
&    AND bitwise
|    OR bitwise
^    XOR bitwise (OR esclusivo)
~    NOT bitwise (complemento)
<<   Shift left
>>   Shift right
```

AND Bitwise (&)

Bit a bit: 1 solo se ENTRAMBI i bit sono 1.

```
int a = 12;  // 1100
int b = 10;  // 1010
int c = a & b;  // 1000 = 8

// Uso: mascherare bit
int flags = 0b11010110;
int bit3 = (flags & 0b00001000) != 0;  // Testa bit 3
```

OR Bitwise (|)

Bit a bit: 1 se ALMENO UNO dei bit è 1.

```
int a = 12;  // 1100
int b = 10;  // 1010
int c = a | b;  // 1110 = 14

// Uso: settare bit
flags = flags | 0b00001000;  // Setta bit 3
// oppure: flags |= 0b00001000;
```

XOR Bitwise (^)

Bit a bit: 1 se i bit sono DIVERSI.

```
int a = 12;  // 1100
int b = 10;  // 1010
int c = a ^ b;  // 0110 = 6

// Uso: toggle bit
flags = flags ^ 0b00001000;  // Inverte bit 3

// Swap senza variabile temp
a = a ^ b;
b = a ^ b;
a = a ^ b;
```

NOT Bitwise (~)

Inverte tutti i bit (complemento a 1).

```c
int a = 12;   // 00001100 (8 bit)
int b = ~a;   // 11110011 = -13 (complemento a 2)
```

Shift Left (<<)

Sposta bit a sinistra (equivale a moltiplicare per 2^n).

```c
int a = 5;      // 00000101
int b = a << 1; // 00001010 = 10 (5 * 2)
int c = a << 2; // 00010100 = 20 (5 * 4)
```

Shift Right (>>)

Sposta bit a destra (equivale a dividere per 2^n).

```c
int a = 20;     // 00010100
int b = a >> 1; // 00001010 = 10 (20 / 2)
int c = a >> 2; // 00000101 = 5  (20 / 4)
```

Nota: Con numeri signed negativi, >> può inserire 1 a sinistra (arithmetic shift) o 0 (logical shift) - dipende dall'implementazione.

Applicazioni Pratiche

Moltiplicazione/divisione veloce per potenze di 2:

```c
x = x << 3;  // x = x * 8  (più veloce di x *= 8)
x = x >> 2;  // x = x / 4  (più veloce di x /= 4)
```

Maschere e flag:

```c
#define FLAG_A 0x01  // 00000001
#define FLAG_B 0x02  // 00000010
#define FLAG_C 0x04  // 00000100

int flags = 0;
flags |= FLAG_A;     // Setta FLAG_A
flags |= FLAG_C;     // Setta FLAG_C

if (flags & FLAG_A) {  // Testa FLAG_A
    printf("FLAG_A è settato\n");
}

flags &= ~FLAG_A;    // Cancella FLAG_A
```

PARTE PRATICA (2 ore)

ESERCIZI GUIDATI

Esercizio 1: Operatori Aritmetici (20 min)

1.1 Calcolatrice completa (programma corretto e completo)

```c
#include <stdio.h>

int main() {
    float a, b;

    printf("Inserisci primo numero: ");
    if (scanf("%f", &a) != 1) return 1;

    printf("Inserisci secondo numero: ");
    if (scanf("%f", &b) != 1) return 1;

    printf("\n=== RISULTATI ===\n");
    printf("%.2f + %.2f = %.2f\n", a, b, a + b);
    printf("%.2f - %.2f = %.2f\n", a, b, a - b);
    printf("%.2f × %.2f = %.2f\n", a, b, a * b);

    // Divisione con controllo per divisione per zero
    if (b != 0.0f) {
        printf("%.2f ÷ %.2f = %.2f\n", a, b, a / b);
    } else {
        printf("%.2f ÷ %.2f = ERRORE (divisione per zero)\n", a, b);
    }

    // Modulo: solo per interi (mostriamo conversione esplicita)
    {
        int ia = (int)a;
        int ib = (int)b;
        if (ib != 0) {
            printf("%d %% %d = %d\n", ia, ib, ia % ib);
        } else {
            printf("%d %% %d = ERRORE (divisione per zero)\n", ia, ib);
        }
    }

    return 0;
}
```

1.2 Conversione tempo:

```c
#include <stdio.h>

int main() {
    int secondi_totali;
    int ore, minuti, secondi;

    printf("Inserisci secondi totali: ");
    scanf("%d", &secondi_totali);

    ore = secondi_totali / 3600;
    minuti = (secondi_totali % 3600) / 60;
    secondi = secondi_totali % 60;

    printf("%d secondi = %d ore, %d minuti, %d secondi\n",
           secondi_totali, ore, minuti, secondi);

    return 0;
}
```

Esempio output:

```
Inserisci secondi totali: 3725
3725 secondi = 1 ore, 2 minuti, 5 secondi
```

1.3 Divisione intera vs reale:

```c
#include <stdio.h>

int main() {
    int dividendo, divisore;

    printf("Dividendo: ");
    scanf("%d", &dividendo);

    printf("Divisore: ");
    scanf("%d", &divisore);

    if (divisore == 0) {
        printf("ERRORE: Divisione per zero!\n");
        return 1;
    }

    int quoziente = dividendo / divisore;
    int resto = dividendo % divisore;
    float risultato_reale = (float)dividendo / divisore;

    printf("\n=== DIVISIONE INTERA ===\n");
    printf("%d ÷ %d = %d (quoziente)\n", dividendo, divisore, quoziente);
    printf("%d %% %d = %d (resto)\n", dividendo, divisore, resto);

    printf("\n=== DIVISIONE REALE ===\n");
    printf("%d ÷ %d = %.4f\n", dividendo, divisore, risultato_reale);

    // Verifica
    printf("\nVerifica: %d × %d + %d = %d\n",
           quoziente, divisore, resto, quoziente * divisore + resto);

    return 0;
}
```

Esercizio 2: Incremento e Decremento (20 min)

2.1 Differenza pre/post incremento:

```c
#include <stdio.h>

int main() {
    int x = 5, y = 5;

    printf("Valori iniziali: x = %d, y = %d\n\n", x, y);

    // Post-incremento
    printf("Post-incremento (x++):\n");
    printf("Valore di x++: %d\n", x++);  // Stampa 5
    printf("Valore di x dopo: %d\n\n", x);  // Ora è 6

    // Pre-incremento
    printf("Pre-incremento (++y):\n");
    printf("Valore di ++y: %d\n", ++y);  // Stampa 6
    printf("Valore di y dopo: %d\n", y);  // Ancora 6

    return 0;
}
```

Output:

```
Valori iniziali: x = 5, y = 5

Post-incremento (x++):
Valore di x++: 5
Valore di x dopo: 6

Pre-incremento (++y):
Valore di ++y: 6
Valore di y dopo: 6
```

2.2 Contatore con incremento:

```c
#include <stdio.h>

int main() {
    int contatore = 0;

    printf("Contatore iniziale: %d\n", contatore);

    contatore++;
    printf("Dopo contatore++: %d\n", contatore);

    ++contatore;
    printf("Dopo ++contatore: %d\n", contatore);

    contatore += 5;
    printf("Dopo contatore += 5: %d\n", contatore);

    contatore--;
    printf("Dopo contatore--: %d\n", contatore);

    return 0;
}
```

2.3 Esercizio pratico - conto alla rovescia:

```c
#include <stdio.h>

int main() {
    int n;

    printf("Inserisci numero per conto alla rovescia: ");
    scanf("%d", &n);

    printf("\nConto alla rovescia:\n");
    while (n > 0) {
        printf("%d...\n", n);
        n--;  // o --n, stesso effetto qui
    }
    printf("ZERO!\n");

    return 0;
}
```

Esercizio 3: Operatori Relazionali e Logici (25 min)

3.1 Controllo età per cinema:

```c
#include <stdio.h>

int main() {
    int eta;
    char tessera;

    printf("Inserisci età: ");
    scanf("%d", &eta);

    printf("Hai tessera fedeltà? (S/N): ");
    scanf(" %c", &tessera);

    float prezzo = 10.0;  // Prezzo base

    printf("\n=== CALCOLO BIGLIETTO ===\n");

    // Bambini (< 12) e anziani (> 65): sconto 50%
    if (eta < 12 || eta > 65) {
        prezzo = prezzo * 0.5;
        printf("Sconto età: -50%%\n");
    }

    // Sconto tessera: ulteriore 10%
    if (tessera == 'S' || tessera == 's') {
        prezzo = prezzo * 0.9;
        printf("Sconto tessera: -10%%\n");
    }

    printf("Prezzo finale: %.2f€\n", prezzo);

    return 0;
}
```

3.2 Validazione password (semplice):

```c
#include <stdio.h>

int main() {
    int lunghezza;
    char ha_maiuscola, ha_numero;

    printf("=== VERIFICA PASSWORD ===\n");
    printf("Lunghezza password: ");
    scanf("%d", &lunghezza);

    printf("Contiene almeno una maiuscola? (S/N): ");
    scanf(" %c", &ha_maiuscola);

    printf("Contiene almeno un numero? (S/N): ");
    scanf(" %c", &ha_numero);

    // Password valida se:
    // - Lunghezza >= 8 E
    // - Ha maiuscola E
    // - Ha numero
    int valida = (lunghezza >= 8) &&
                 (ha_maiuscola == 'S' || ha_maiuscola == 's') &&
                 (ha_numero == 'S' || ha_numero == 's');

    printf("\n");
    if (valida) {
        printf("✓ Password VALIDA\n");
    } else {
        printf("✗ Password NON VALIDA\n");

        if (lunghezza < 8) {
            printf("  - Deve essere almeno 8 caratteri\n");
        }
        if (ha_maiuscola != 'S' && ha_maiuscola != 's') {
            printf("  - Deve contenere una maiuscola\n");
        }
        if (ha_numero != 'S' && ha_numero != 's') {
            printf("  - Deve contenere un numero\n");
        }
    }

    return 0;
}
```

3.3 Controllo triangolo (dati 3 lati):

```c
#include <stdio.h>

int main() {
    float a, b, c;

    printf("Inserisci i tre lati del triangolo:\n");
    printf("Lato 1: ");
    scanf("%f", &a);
    printf("Lato 2: ");
    scanf("%f", &b);
    printf("Lato 3: ");
    scanf("%f", &c);

    // Disuguaglianza triangolare:
    // La somma di due lati deve essere > del terzo
    int valido = (a + b > c) && (a + c > b) && (b + c > a);

    if (valido) {
        printf("\n✓ Può formare un triangolo valido\n");

        // Classificazione
        if (a == b && b == c) {
            printf("Tipo: EQUILATERO\n");
        } else if (a == b || b == c || a == c) {
            printf("Tipo: ISOSCELE\n");
        } else {
            printf("Tipo: SCALENO\n");
        }
    } else {
        printf("\n✗ NON può formare un triangolo valido\n");
    }

    return 0;
}
```

3.4 Anno bisestile:

```c
#include <stdio.h>

int main() {
    int anno;

    printf("Inserisci anno: ");
    scanf("%d", &anno);

    // Regole anno bisestile:
    // - Divisibile per 4 E
    // - (NON divisibile per 100 O divisibile per 400)

    int bisestile = (anno % 4 == 0) &&
                    ((anno % 100 != 0) || (anno % 400 == 0));

    if (bisestile) {
        printf("%d è BISESTILE (366 giorni)\n", anno);
    } else {
        printf("%d NON è bisestile (365 giorni)\n", anno);
    }

    return 0;
}
```

Esercizio 4: Operatori Composti e Ternario (20 min)

4.1 Uso operatori composti:

```c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Valore iniziale: x = %d\n\n", x);

    x += 5;
    printf("Dopo x += 5:  x = %d\n", x);

    x -= 3;
    printf("Dopo x -= 3:  x = %d\n", x);

    x *= 2;
    printf("Dopo x *= 2:  x = %d\n", x);

    x /= 4;
    printf("Dopo x /= 4:  x = %d\n", x);

    x %= 5;
    printf("Dopo x %%= 5:  x = %d\n", x);

    return 0;
}
```

Output:

```
Valore iniziale: x = 10

Dopo x += 5:  x = 15
Dopo x -= 3:  x = 12
Dopo x *= 2:  x = 24
Dopo x /= 4:  x = 6
Dopo x %= 5:  x = 1
```

4.2 Operatore ternario - esempi pratici:

```c
#include <stdio.h>

int main() {
    int a = 15, b = 20;

    // Massimo tra due numeri
    int max = (a > b) ? a : b;
    printf("Massimo tra %d e %d: %d\n", a, b, max);

    // Minimo tra due numeri
    int min = (a < b) ? a : b;
    printf("Minimo tra %d e %d: %d\n", a, b, min);

    // Valore assoluto
    int x = -7;
    int abs_x = (x < 0) ? -x : x;
    printf("Valore assoluto di %d: %d\n", x, abs_x);

    // Pari o dispari
    int n = 13;
    printf("%d è %s\n", n, (n % 2 == 0) ? "pari" : "dispari");

    // Maggiorenne
    int eta = 17;
    printf("Sei %s\n", (eta >= 18) ? "maggiorenne" : "minorenne");

    return 0;
}
```

4.3 Calcolo sconto con ternario:

```c
#include <stdio.h>

int main() {
    float importo;

    printf("Inserisci importo spesa: ");
    scanf("%f", &importo);

    // Sconto: 10% se >= 100€, altrimenti 5%
    float percentuale_sconto = (importo >= 100.0) ? 10.0 : 5.0;
    float sconto = importo * percentuale_sconto / 100.0;
    float totale = importo - sconto;

    printf("\n=== RIEPILOGO ===\n");
    printf("Importo:   %.2f€\n", importo);
    printf("Sconto:    %.0f%% (%.2f€)\n", percentuale_sconto, sconto);
    printf("Totale:    %.2f€\n", totale);

    return 0;
}
```

Esercizio 5: Progetti Completi (35 min)

5.1 Calcolatore BMI (Body Mass Index):

```c
#include <stdio.h>

int main() {
    float peso, altezza, bmi;

    printf("=== CALCOLO BMI ===\n");
    printf("Peso (kg): ");
    scanf("%f", &peso);

    printf("Altezza (m): ");
    scanf("%f", &altezza);

    // Validazione
    if (peso <= 0 || altezza <= 0) {
        printf("ERRORE: Valori non validi!\n");
        return 1;
    }

    bmi = peso / (altezza * altezza);

    printf("\n=== RISULTATO ===\n");
    printf("BMI: %.1f\n", bmi);

    // Classificazione OMS
    printf("Categoria: ");
    if (bmi < 16.0) {
        printf("Grave magrezza\n");
    } else if (bmi < 18.5) {
        printf("Sottopeso\n");
    } else if (bmi < 25.0) {
        printf("Normopeso\n");
    } else if (bmi < 30.0) {
        printf("Sovrappeso\n");
    } else if (bmi < 35.0) {
        printf("Obesità classe 1\n");
    } else if (bmi < 40.0) {
        printf("Obesità classe 2\n");
    } else {
        printf("Obesità classe 3\n");
    }

    return 0;
}
```

5.2 Gioco indovina numero (con tentativi):

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int numero_segreto, tentativo, contatore = 0;
    int max_tentativi = 7;

    // Genera numero casuale tra 1 e 100
    srand(time(NULL));
    numero_segreto = rand() % 100 + 1;

    printf("=== INDOVINA IL NUMERO ===\n");
    printf("Ho pensato un numero tra 1 e 100.\n");
    printf("Hai %d tentativi.\n\n", max_tentativi);

    while (contatore < max_tentativi) {
        contatore++;
        printf("Tentativo %d/%d: ", contatore, max_tentativi);
        scanf("%d", &tentativo);

        if (tentativo == numero_segreto) {
            printf("\n BRAVO! Hai indovinato in %d tentativi!\n", contato
            return 0;
        } else if (tentativo < numero_segreto) {
            printf("Troppo basso!\n");
        } else {
            printf("Troppo alto!\n");
        }

        // Suggerimenti extra agli ultimi tentativi
        if (contatore >= max_tentativi - 2) {
            int diff = (tentativo > numero_segreto) ?
                       tentativo - numero_segreto :
                       numero_segreto - tentativo;

            if (diff <= 5) {
                printf("(Sei molto vicino!)\n");
            } else if (diff <= 10) {
                printf("(Sei vicino!)\n");
            }
        }

        printf("\n");
    }

    printf(" Hai esaurito i tentativi!\n");
    printf("Il numero era: %d\n", numero_segreto);

    return 0;
}
```

5.3 Calcolatore tasse (aliquote progressive):

```c
#include <stdio.h>

int main() {
    float reddito, tasse = 0.0;

    printf("=== CALCOLO TASSE (sistema semplificato) ===\n");
    printf("Inserisci reddito annuo (€): ");
    scanf("%f", &reddito);

    if (reddito <= 0) {
        printf("Reddito non valido!\n");
        return 1;
    }

    // Aliquote progressive (esempio semplificato)
    // 0 - 15000: 23%
    // 15001 - 28000: 27%
    // 28001 - 55000: 38%
    // oltre 55000: 43%

    if (reddito <= 15000) {
        tasse = reddito * 0.23;
    } else if (reddito <= 28000) {
        tasse = 15000 * 0.23 + (reddito - 15000) * 0.27;
    } else if (reddito <= 55000) {
        tasse = 15000 * 0.23 + 13000 * 0.27 + (reddito - 28000) * 0.38;
    } else {
        tasse = 15000 * 0.23 + 13000 * 0.27 + 27000 * 0.38 +
                (reddito - 55000) * 0.43;
    }

    float netto = reddito - tasse;
    float aliquota_media = (tasse / reddito) * 100.0;

    printf("\n=== RIEPILOGO ===\n");
    printf("Reddito lordo:     %10.2f€\n", reddito);
    printf("Tasse:             %10.2f€ (%.1f%%)\n", tasse, aliquota_media)
    printf("Reddito netto:     %10.2f€\n", netto);

    return 0;
}
```

5.4 Convertitore numeri romani (da decimale):

```c
#include <stdio.h>

int main() {
    int numero, originale;

    printf("Inserisci numero (1-3999): ");
    scanf("%d", &numero);

    if (numero < 1 || numero > 3999) {
        printf("Numero fuori range!\n");
        return 1;
    }

    originale = numero;
    printf("%d in numeri romani: ", originale);

    // Migliaia
    while (numero >= 1000) {
        printf("M");
        numero -= 1000;
    }

    // Centinaia
    if (numero >= 900) {
        printf("CM");
```

```c
        printf("CM");
        numero -= 900;
    }
    if (numero >= 500) {
        printf("D");
        numero -= 500;
    }
    if (numero >= 400) {
        printf("CD");
        numero -= 400;
    }
    while (numero >= 100) {
        printf("C");
        numero -= 100;
    }

    // Decine
    if (numero >= 90) {
        printf("XC");
        numero -= 90;
    }
    if (numero >= 50) {
        printf("L");
        numero -= 50;
    }
    if (numero >= 40) {
        printf("XL");
        numero -= 40;
    }
    while (numero >= 10) {
        printf("X");
        numero -= 10;
    }

    // Unità
    if (numero >= 9) {
        printf("IX");
        numero -= 9;
    }
    if (numero >= 5) {
        printf("V");
        numero -= 5;
    }
    if (numero >= 4) {
        printf("IV");
        numero -= 4;
    }
    while (numero >= 1) {
        printf("I");
        numero -= 1;
    }

    printf("\n");

    return 0;
}
```

Esempio output:

```
Inserisci numero (1-3999): 1994
1994 in numeri romani: MCMXCIV
```

ESERCIZI AUTONOMI

Esercizio 6: Pratica Libera

6.1 Crea un programma che verifica se un numero è primo (usa %, ciclo anticipato).

6.2 Programma che calcola il fattoriale di un numero (usa * e ciclo).

6.3 Convertitore binario → decimale (inserisci cifre una a una).

6.4 Calcolatore interesse composto bancario.

6.5 Gioco "Morra cinese" (sasso, carta, forbice) contro computer.

ESERCIZI DA SVOLGERE A CASA

Homework 1: Crea un programma "Analisi numero" che: - Legge un numero intero - Dice se è pari/dispari - Calcola valore assoluto - Dice se è multiplo di 3, 5, o 10 - Conta le cifre - Calcola somma delle cifre

Homework 2: Sistema di voto esami: - Chiede voti di 5 esami (0-30) - Calcola media - Verifica se tutti i voti sono >= 18 - Dice se studente è promosso - Calcola media ponderata se forniti i crediti

Homework 3: Calcolatore distanza punti: - Chiede coordinate (x1, y1) e (x2, y2) - Calcola distanza euclidea: $\sqrt{[(x2-x1)^2 + (y2-y1)^2]}$ - Usa solo operatori visti (no sqrt() direttamente, ma x*x per quadrato)

Homework 4: Gioco "Divisione veloce": - Mostra due numeri casuali - Utente deve rispondere velocemente se primo divisibile per secondo - Conta risposte corrette/sbagliate - 10 domande

Homework 5: Esperimenti con operatori bitwise: - Chiedi un numero - Mostra rappresentazione binaria (usa shift e &) - Conta bit a 1 - Testa/setta/cancella bit specifici - Fai swap con XOR

Homework 6: Mini-linguaggio matematico: - Chiedi espressione tipo: "5 + 3 * 2" - Leggi numero, operatore, numero - Valuta rispettando precedenza - Gestisci errori (divisione per zero)

APPROFONDIMENTI

Sequence Points e Undefined Behavior

Sequence point: Punto dove tutte le valutazioni precedenti sono complete.

```c
int x = 5;
int y = x++ + ++x;  // UNDEFINED BEHAVIOR!
// Non si sa quale ++ viene valutato prima
```

Sequence points comuni: - Fine istruzione (`;`) - Prima di chiamata funzione - `&&`, `||`, `?:`, `,` (valutazione da sinistra)

Regola d'oro: Non modificare stessa variabile più volte tra sequence points.

```c
// ✗ MALE
i = i++;
a[i] = i++;
f(i++, i++);

// ✓ BENE
i++;
a[i] = value; i++;
int temp1 = i++; int temp2 = i++; f(temp1, temp2);
```

Short-Circuit Evaluation

Ottimizzazione: `&&` e `||` non valutano secondo operando se non necessario.

```c
// Se x == 0, NON valuta (y / x)
if (x != 0 && (y / x) > 2) { }

// Se p != NULL, SOLO ALLORA deref *p
if (p != NULL && *p == 'A') { }
```

Attenzione effetti collaterali:

```c
if (x > 0 || ++contatore > 10) { }
// Se x > 0, contatore NON viene incrementato!
```

Operatori Bitwise Avanzati

Maschere comuni:

```c
// Setta bit n
x |= (1 << n);

// Cancella bit n
x &= ~(1 << n);

// Toggl bit n
x ^= (1 << n);

// Testa bit n
if (x & (1 << n)) { }

// Isola bit meno significativo a 1
x & -x

// Cancella bit meno significativo a 1
x & (x - 1)
```

Conta bit a 1 (popcount):

```c
int count = 0;
while (x) {
    count++;
    x &= (x - 1);  // Cancella bit meno significativo
}
```

Ottimizzazioni Compilatore

Il compilatore può **ottimizzare** espressioni:

```c
// Codice originale
int x = 2 + 3;

// Dopo ottimizzazione
int x = 5;  // Calcolato a compile-time
```

Constant folding:

```c
int y = 10 * 20 / 5;  // Diventa: int y = 40;
```

Strength reduction:

```c
x * 2    → x << 1    // Shift più veloce
x / 4    → x >> 2
```

Dead code elimination:

```c
if (0) {
    // Questo codice viene rimosso dal compilatore
}
```

Promozione Intera Implicita

Integer promotion: char e short promossi automaticamente a int.

```c
char a = 100, b = 100;
char c = a + b;  // Overflow! a e b promossi a int, risultato 200,
                 // poi troncato a char (overflow)

printf("%d\n", c);  // Probabilmente: -56 (256 - 200)
```

Usual arithmetic conversions:

```c
int i;
float f;
double d;

i + f  // i promosso a float
f + d  // f promosso a double
```

RIEPILOGO CONCETTI CHIAVE

Operatori Aritmetici

```
+  addizione
-  sottrazione
*  moltiplicazione
/  divisione (intera se operandi int)
%  modulo (solo interi)
```

Incremento/Decremento

```
++x  pre-incremento (prima incrementa, poi usa)
x++  post-incremento (prima usa, poi incrementa)
--x  pre-decremento
x--  post-decremento
```

Operatori Relazionali

```
==  uguale
!=  diverso
>   maggiore
<   minore
>=  maggiore o uguale
<=  minore o uguale
```

Operatori Logici

```
&&  AND (entrambi veri)
||  OR (almeno uno vero)
!   NOT (inverti)
```

Operatori Composti

```
+=  -=  *=  /=  %=
x += 5  equivale a  x = x + 5
```

Operatore Ternario

```
condizione ? se_vero : se_falso
max = (a > b) ? a : b;
```

TABELLE DI RIFERIMENTO RAPIDO

Precedenza Operatori (Comuni)

Precedenza	Operatori	Associatività
Più alta	() [] . ->	L→R
	! ~ ++ – + - (unari)	R→L
	* / %	L→R
	+ -	L→R
	< <= > >=	L→R
	== !=	L→R
	&&	L→R
	||	L→R
	?:	R→L
	= += -= *= /= %=	R→L
Più bassa	,	L→R

Tabelle Verità

AND (&&):

```
A  B  A&&B
0  0   0
0  1   0
1  0   0
1  1   1
```

OR (||):

```
A  B  A||B
0  0   0
0  1   1
1  0   1
1  1   1
```

NOT (!):

```
A  !A
0   1
1   0
```

PROBLEMI COMUNI E SOLUZIONI

Problema 1: = invece di ==

```c
int x = 5;
if (x = 10) {  // BUG! Assegna, non confronta
    printf("Sempre vero!\n");
}
```

Soluzione:

```c
if (x == 10) {  // Corretto: confronto
    printf("OK\n");
}
```

Problema 2: Divisione intera inaspettata

```c
int a = 5, b = 2;
float result = a / b;  // result = 2.0 (non 2.5!)
```

Soluzione:

```c
float result = (float)a / b;  // 2.5
// oppure
float result = a / (float)b;
// oppure
float result = (float)(a) / (float)(b);
```

Problema 3: Modulo con negativi

```c
int a = -7 % 3;  // Risultato dipende da implementazione!
// Può essere -1 o 2
```

Soluzione: Usa solo con numeri positivi, o gestisci manualmente:

```c
int mod(int a, int b) {
    int r = a % b;
    return (r < 0) ? r + b : r;
}
```

Problema 4: Overflow senza warning

```c
int x = 2000000000;
int y = x + 1000000000;  // Overflow silenzioso!
printf("%d\n", y);  // Valore negativo!
```

Soluzione: Usa tipo più grande o controlla limiti:

```c
#include <limits.h>
if (x > INT_MAX - 1000000000) {
    printf("Overflow!\n");
} else {
    int y = x + 1000000000;
}
```

Problema 5: Float comparison

```c
float a = 0.1 + 0.2;
if (a == 0.3) {  // Probabilmente falso!
    printf("Uguale\n");
}
```

Soluzione:

```c
#include <math.h>
#define EPSILON 0.00001
if (fabs(a - 0.3) < EPSILON) {
    printf("Uguale (entro tolleranza)\n");
}
```

Problema 6: Precedenza confusa

```c
int x = 10;
if (x > 5 && x < 20 || x == 0) { }
// Viene valutato come: (x > 5 && x < 20) || (x == 0)
// Ma potresti voler dire: x > 5 && (x < 20 || x == 0)
```

Soluzione: Usa parentesi esplicite:

```c
if ((x > 5 && x < 20) || x == 0) { }  // Chiaro
```

Problema 7: Incremento multiplo

```c
int i = 5;
int x = i++ + i++ + i++;  // UNDEFINED BEHAVIOR!
```

Soluzione: Usa istruzioni separate:

```c
int x = i;
i++;
x += i;
i++;
x += i;
i++;
```

ESERCIZI BONUS AVANZATI

Bonus 1: Swap XOR (senza temp)

```c
#include <stdio.h>

int main() {
    int a = 5, b = 10;

    printf("Prima: a=%d, b=%d\n", a, b);

    // Swap con XOR (solo per curiosità)
    a = a ^ b;
    b = a ^ b;  // b diventa a originale
    a = a ^ b;  // a diventa b originale

    printf("Dopo:  a=%d, b=%d\n", a, b);

    return 0;
}
```

Bonus 2: Controllo potenza di 2

```c
#include <stdio.h>

int main() {
    int n;

    printf("Inserisci numero: ");
    scanf("%d", &n);

    // Un numero è potenza di 2 se ha un solo bit a 1
    // Trucco: n & (n-1) == 0 per potenze di 2
    if (n > 0 && (n & (n - 1)) == 0) {
        printf("%d è una potenza di 2\n", n);
    } else {
        printf("%d NON è una potenza di 2\n", n);
    }

    return 0;
}
```

Bonus 3: Conta bit a 1

```c
#include <stdio.h>

int main() {
    unsigned int n;
    int count = 0;

    printf("Inserisci numero: ");
    scanf("%u", &n);

    unsigned int temp = n;
    while (temp) {
        count += temp & 1;  // Aggiunge 1 se bit meno significativo è 1
        temp >>= 1;         // Shift a destra
    }

    printf("Il numero %u ha %d bit a 1\n", n, count);

    return 0;
}
```

Bonus 4: Inversione cifre

```c
#include <stdio.h>

int main() {
    int n, invertito = 0;

    printf("Inserisci numero: ");
    scanf("%d", &n);

    int originale = n;

    while (n > 0) {
        int cifra = n % 10;           // Estrae ultima cifra
        invertito = invertito * 10 + cifra;  // Aggiunge a sinistra
        n /= 10;                      // Rimuove ultima cifra
    }

    printf("%d invertito è %d\n", originale, invertito);

    return 0;
}
```

Esempio:

```
Inserisci numero: 1234
1234 invertito è 4321
```

GLOSSARIO TECNICO

- **Operatore**: Simbolo che esegue operazione (+, -, *, /, etc.)
- **Operando**: Valore su cui opera un operatore
- **Espressione**: Combinazione di operatori e operandi che produce valore
- **Precedenza**: Ordine di valutazione degli operatori
- **Associatività**: Direzione valutazione (L→R o R→L)
- **Short-circuit**: Valutazione parziale (&&, ||)
- **L-value**: Espressione che può stare a sinistra di =
- **R-value**: Espressione che può stare solo a destra di =
- **Side effect**: Modifica stato (come ++, –)
- **Sequence point**: Punto dove valutazioni sono complete
- **Undefined behavior**: Comportamento non specificato dallo standard
- **Integer promotion**: Conversione automatica char/short → int
- **Overflow**: Risultato supera massimo rappresentabile
- **Underflow**: Risultato inferiore a minimo rappresentabile

- **Modulo**: Resto divisione intera (%)
- **Bitwise**: Operazione bit a bit (&, |, ^, ~, <<, >>)

PREPARAZIONE PER LA PROSSIMA LEZIONE

Nella prossima lezione useremo questi operatori dentro i **costrutti di selezione** (if, else, switch) per creare programmi che prendono decisioni complesse. È qui che i programmi diventano veramente intelligenti e interattivi!

Gli operatori che hai imparato oggi sono gli **strumenti base** che userai in ogni singolo programma C che scriverai. Più pratichi, più diventeranno naturali come scrivere in italiano.